

JavaScript is a high-level, interpreted scripting language used to create dynamic and interactive web pages. It is mainly used in web development to add functionality to websites. JavaScript works with HTML and CSS to design complete web applications. JavaScript runs directly inside the web browser and allows developers to modify webpage content without reloading the page. It supports features such as event handling, form validation, animations, and dynamic content updates. JavaScript can also be used for server-side development with Node.js.

Because of its flexibility and cross-platform support, JavaScript has become one of the most widely used programming languages for building modern web applications.

Features of JavaScript

- **It is a lightweight scripting language.**
- **It is an interpreted language (no compilation required).**
- **It supports event-based programming.**
- **It is used to create dynamic and interactive web pages.**

Example

```
let name = "Student";  
alert("Hello " + name);
```

Output: A message box displays Hello Student.

JavaScript Overview & Basics

Definition : JavaScript is a high-level, interpreted programming language used to create interactive web pages. It runs inside the web browser and allows dynamic behavior on websites.

Key Features

- 1. Client-side scripting**
 - **Runs in the browser without server interaction.**
- 2. Interpreted language**
 - **Code executes line by line without compilation.**
- 3. Dynamic typing**
 - **Variables do not require a specific data type.**
- 4. Event-driven**
 - **Executes code when an event occurs (click, hover).**
- 5. Platform independent**
 - **Works on all major browsers.**

Basic Syntax Example

```
<script>  
let name = "Rahul";  
alert("Hello " + name);  
</script>
```

Real-life Example

When a user clicks a login button, JavaScript checks whether the username and password fields are empty before sending data to the server.

1. Variables in JavaScript

Definition : A variable is used to store data values in a program. The value stored in a variable can be changed during program execution.

Types of Variables

1. **Var** - Old way of declaring variables.
 - Function scoped.
2. **Let** - Introduced in ES6.
 - Block scoped and commonly used.
3. **Const** - Used to declare constant values.
 - Value cannot be changed after assignment.

Example

```
let name = "Rahul";  
const age = 21;  
var city = "Rajkot";  
console.log(name);
```

Practical Example

In a registration form, variables store user data like name, email, and password.

2. Conditional Statements in JavaScript

Definition : Conditional statements are used to perform different actions based on conditions.

Types of Conditional Statements

1. **if statement**
2. **if...else statement**
3. **else if statement**
4. **switch statement**

Example (if-else)

```
let marks = 60;  
if(marks >= 50){  
    console.log("Pass");  
}else{  
    console.log("Fail"); }  

```

Practical Example

A website checks user login credentials and displays login success or error message.

3. Loops in JavaScript

Definition : Loops are used to repeat a block of code multiple times until a condition becomes false.

Types of Loops

1. **for loop**
2. **while loop**
3. **do...while loop**

Example 1: for Loop

```
for(let i = 1; i <= 5; i++){  
  console.log(i); }
```

Explanation

This loop prints numbers from 1 to 5.

Example 2: while Loop

```
let i = 1;  
while(i <= 3){  
  console.log(i);  
  i++; }
```

Example 3: do...while Loop

```
let i = 1;  
do{console.log(i);  
  i++;  
}while(i <= 3);
```

Express JS

Definition : Express JS is a web application framework for Node.js used to build web applications and RESTful APIs easily and quickly.

It provides tools for routing, middleware, and server management.

Features of Express JS

1. Fast and Lightweight

- Requires minimal code to create a web server.

2. Routing Support

- Handles different URLs and HTTP requests such as GET and POST.

3. Middleware Support

- Allows execution of functions between request and response.

4. Template Engine Support

- Supports template engines like EJS, Pug, Handlebars.

5. Easy API Development

- Used for building REST APIs for web and mobile applications.

Installing Express JS

Step 1: Install Node.js

Step 2: Install Express using npm

npm install express

Creating a Simple Express Server

Example

```
const express = require("express");  
const app = express();  
app.listen(3000, () => {  
  console.log("Server is running on port 3000");  
});
```

Definition : ES6 (ECMAScript 2015) is an updated version of JavaScript that introduced many new features to make code simpler, cleaner, and more powerful. It improves readability, performance, and modern programming support.

Key Features of ES6

1. let and const

ES6 introduced let and const for declaring variables.

- **let** → value can be changed
- **const** → value cannot be changed

Example

```
let age = 20;  
age = 25;  
const country = "India";
```

2. Arrow Functions

Arrow functions provide a shorter syntax for writing functions.

Example

```
const add = (a,b) => {  
    return a + b; }  
console.log(add(5,3));
```

Advantage

- **Less code**
- **More readable**

3. Template Literals

Template literals allow embedding variables inside strings using backticks (`).

Example

```
let name = "Rahul";  
console.log(`Hello ${name}`);
```

Advantage

Makes string formatting easier.

4. Default Parameters : Functions can have default values for parameters.

Example

```
function greet(name = "Guest"){  
    console.log("Hello " + name); }  
greet();
```

5. Modules : ES6 allows exporting and importing code between files.

Example

```
Export  
export const pi = 3.14;  
Import  
import {pi} from "./file.js";
```

3. Advantages of ES6

- **Cleaner and shorter code**
- **Better support for modern web development**
- **Improved readability and maintainability**
- **Supports object-oriented programming features**

1. Functions in JavaScript

Definition : A function is a block of code that performs a specific task. It is executed when the function is called or invoked.

Advantages

- **Code reuse**
- **Reduces repetition**
- **Makes program organized**

Syntax

```
function functionName(parameters){    // code to execute    }
```

Example

```
function add(a, b){  
    return a + b; }  
console.log(add(5,3));
```

Explanation

The function add() takes two numbers and returns their sum

2. Arrays in JavaScript

Definition : An array is a data structure used to store multiple values in a single variable.

Syntax : let arrayName = [value1, value2, value3];

Example :

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits[0]);
```

Explanation

- **fruits[0] → Apple**
- **fruits[1] → Banana**

Common Array Methods

- **push() → Add element**
- **pop() → Remove last element**
- **length → Count number of elements**

Example

```
fruits.push("Orange");  
console.log(fruits.length);
```

3. Events in JavaScript

Definition : An event is an action that occurs in the browser, such as clicking a button, pressing a key, or loading a page.

Common Events :

onclick – when user clicks an element

onmouseover – when mouse moves over element

onkeypress – when a key is pressed

onsubmit – when a form is submitted

Example

```
<button  
onclick="showMessage()">Click  
Me</button> <script>  
function showMessage(){  
    alert("Button Clicked"); }  
</script>
```

1. ES6 Classes

Definition : An ES6 class is a blueprint used to create objects in JavaScript. It supports Object-Oriented Programming (OOP) concepts like properties and methods.

Syntax

```
class ClassName {  
  constructor(parameters){  
    this.property = value; }  
  methodName(){ // code } }
```

Example

```
class Student{  
  constructor(name, age){  
    this.name = name;  
    this.age = age; }  
  display(){  
    console.log(this.name + " " + this.age); } }  
let s1 = new Student("Rahul",21);  
s1.display();
```

2. ES6 Functions (Arrow Functions)

Definition : Arrow functions are a short and modern way of writing functions in JavaScript.

Syntax

```
const functionName = (parameters) => { // code }
```

Example

```
const multiply = (a,b) => {  
  return a * b; }  
console.log(multiply(4,5));
```

Advantages

- **Short syntax**
- **Improves code readability**
- **Commonly used in React applications**

3. Promises in ES6

Definition : A Promise is used to handle asynchronous operations in JavaScript such as API calls, database requests, or file loading.

States of a Promise

1. **Pending** – operation is in progress
2. **Resolved (Fulfilled)** – operation completed successfully
3. **Rejected** – operation failed

Syntax

```
let promise = new Promise(function(resolve, reject){  
  let success = true;  
  if(success){  
    resolve("Operation Successful");  
  }else{  
    reject("Operation Failed");  
  }  
});
```

Using Promise

```
promise.then(function(result){  
  console.log(result);  
}).catch(function(error){  
  console.log(error);  
});
```

Setting up an app with ExpressJS, Routing in ExpressJS, Connecting views with templates, configurations and error handling.1

1. Setting up an Application with ExpressJS

Steps to create an Express application.

Step 1: Install Node.js and Express

npm install express

Step 2: Create Express Server

```
const express = require("express");
const app = express();
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Explanation

- **express()** creates an Express application.
- **app.listen()** starts the server.

2. Routing in ExpressJS

Routing defines how the server responds to different client requests such as GET, POST, PUT, and DELETE.

Example

```
app.get("/", (req, res) => {
  res.send("Home Page");
});
app.get("/about", (req, res) => {
  res.send("About Page");
});
```

Explanation

- **/** → **Home page route**
- **/about** → **About page route**

3. Connecting Views with Templates

Template engines are used to generate dynamic HTML pages.

Common template engines:

- **EJS**
- **Pug**
- **Handlebars**

Example (Using EJS)

Install EJS

npm install ejs

Set template engine

```
app.set("view engine", "ejs");
app.get("/", (req, res) => {
  res.render("index");
});
```

Explanation

The server renders the index.ejs file from the views folder.

Setting up an app with ExpressJS, Routing in ExpressJS, Connecting views with templates, configurations and error handling. 2

4. Configuration in ExpressJS

Configuration is used to set application settings.

Example

```
app.set("port",3000);  
app.set("view engine","ejs");
```

Explanation

- **port** → **server port number**
- **view engine** → **template engine used**

5. Error Handling in ExpressJS

Error handling manages invalid routes or server errors.

Example (404 Error)

```
app.use((req,res)=>{  
  res.status(404).send("Page Not Found");  
});
```

Explanation

If the user requests a route that does not exist, the server returns **Page Not Found**.